Лабораторная работа №2.
Параллельное вычисление определённого интеграла методом средних прямоугольников.

Выполнила: Колодезникова Софья, 11-400

Описание работы

В работе на языке C реализованы последовательная и параллельная версии вычисления определённого интеграла функции $\sqrt{2 + \sin x}$ на отрезке $[0; \frac{\pi}{2}]$ методом средних прямоугольников, а также режим вычисления с заданной точностью путём удвоения числа разбиений. Проведены три эксперимента: зависимость времени от числа потоков ($n = 10^8$), зависимость значения интеграла и времени от n (8 потоков) и поиск n для точности 10^{-10}. Замеры выполнены на процессоре Intel Core Ultra 5 125H (14 ядер, 18 логических процессоров), время измерялось функцией omp_get_wtime().

Результаты экспериментов

Эксперимент 1. Влияние количества потоков при $n = 10^8$.
Время в секундах последовательной версии: 0,7746 с (среднее по 5 запускам)

Потоки	Время, с	Ускорение
1	0,7420	1,04
2	0,4840	1,60
4	0,2680	2,89
8	0,2280	3,40
16	0,1760	4,40

Эксперимент 2. Влияние числа разбиений при 8 потоках.

n	Значение интеграла	Время, с
10^4	2,546097850401923	<0,001
10^5	2,546097850042073	<0,001
10^6	2,546097850038477	0,004
10^7	2,546097850038420	0,023
10^8	2,546097850038313	0,221

Эксперимент 3. Вычисление с заданной точностью (10^{-10}).

Число разбиений: n = 64000
Время выполнения последовательной версии: 0,0020 с
Время выполнения параллельной версии: 0,0020 с

Вывод

С увеличением числа потоков время выполнения при $n = 10^8$ уменьшалось: с 0,7746 секунд у последовательной версии до 0,1760 с на 16 потоках. Ускорение при этом росло медленнее: 1,60 на двух потоках, 2,89 на четырёх, 3,40 на восьми и 4,40 на шестнадцати.

Число разбиений напрямую влияет на точность: значение интеграла стабилизируется по мере роста n, и уже при $n = 10^6$ совпадают первые 10 знаков после запятой.

Заданная точность 10^{-10} достигнута при n = 64000. Директива reduction здесь эффективна, потому что значения функции в разных точках считаются независимо, а нужна только их общая сумма: каждый поток накапливает свою частичную сумму, и они складываются один раз в конце. Это избавляет от блокировок и гонок данных, которые возникли бы при общей переменной sum.